arXiv:2404.14619v2 [cs.CL] 2 May 2024

OpenELM: An Efficient Language Model Family with Open Training and Inference Framework

Sachin Mehta Mohammad Hossein Sekhavat Qingqing Cao Maxwell Horton
Yanzi Jin Chenfan Sun Iman Mirzadeh Mahyar Najibi Dmitry Belenko
Peter Zatloukal Mohammad Rastegari
Apple

Model	Public dataset	Open		Model size	Pre-training tokens	Average acc. (in %)
		Code	Weights			
OPT [55]	✗	✓	✓	1.3 B	0.2 T	41.49
PyThia [5]	✓	✓	✓	1.4 B	0.3 T	41.83
MobiLlama [44]	✓	✓	✓	1.3 B	1.3 T	43.55
OLMo [17]	✓	✓	✓	1.2 B	3.0 T	43.57
OpenELM (Ours)	✓	✓	✓	1.1 B	1.5 T	**45.93**

Table 1. **OpenELM vs. public LLMs.** OpenELM outperforms comparable-sized existing LLMs pretrained on publicly available datasets. Notably, OpenELM outperforms the recent open LLM, OLMo, by 2.36% while requiring 2× fewer pre-training tokens. The average accuracy is calculated across multiple tasks listed in Tab. 3b, which are also part of the OpenLLM leaderboard [4]. Models pretrained with less data are highlighted in gray color.

Abstract

The reproducibility and transparency of large language models are crucial for advancing open research, ensuring the trustworthiness of results, and enabling investigations into data and model biases, as well as potential risks. To this end, we release OpenELM, a state-of-the-art open language model. OpenELM uses a layer-wise scaling strategy to efficiently allocate parameters within each layer of the transformer model, leading to enhanced accuracy. For example, with a parameter budget of approximately one billion parameters, OpenELM exhibits a 2.36% improvement in accuracy compared to OLMo while requiring 2× fewer pre-training tokens.

Diverging from prior practices that only provide model weights and inference code, and pre-train on private datasets, our release includes the complete framework for training and evaluation of the language model on publicly available datasets, including training logs, multiple checkpoints, and pre-training configurations. We also release code to convert models to MLX library for inference and fine-tuning on Apple devices. This comprehensive release aims to empower and strengthen the open research community, paving the way for future open research endeavors.

Our source code along with pre-trained model weights and training recipes is available at https://github.com/apple/corenet. *Additionally, OpenELM models can be found on HuggingFace at:* https://huggingface.co/apple/OpenELM.

1. Introduction

Transformer-based [48] large language models (LLM) are revolutionizing the field of natural language processing [7, 46]. These models are isotropic, meaning that they have the same configuration (*e.g.*, number of heads and feed-forward network dimensions) for each transformer layer. Though such isotropic models are simple, they may not allocate parameters efficiently inside the model.

In this work, we develop and release OpenELM, a family of pre-trained and fine-tuned models on *publicly* available datasets. At the core of OpenELM lies layer-wise scaling [30], enabling *more efficient parameter* allocation across layers. This method utilizes smaller latent dimensions in the attention and feed-forward modules of the transformer layers closer to the input, and gradually widening the layers as they approach the output.

We release the complete framework, encompassing data preparation, training, fine-tuning, and evaluation procedures, alongside multiple pre-trained checkpoints and training logs, to facilitate open research. Importantly, OpenELM outperforms existing open LLMs that are pre-trained using publicly available datasets (Tab. 1). For example, OpenELM with 1.1 billion parameters outperforms OLMo

[17], which has 1.2 billion parameters, by 2.36% while requiring 2× fewer pre-training tokens.

2. Pre-training

This section describes the framework, including model architecture (§2.1), pre-training data (§2.2), training hyper-parameters (§2.3), and evaluation (§2.4).

2.1. OpenELM architecture

We adopt the decoder-only transformer-based architecture. Following state-of-the-art LLMs, we: (1) do not use learnable bias parameters in any fully-connected (*a.k.a.*, linear) layers, (2) apply pre-normalization using RMSNorm [53] and also, use rotatory positional embedding (ROPE) [43] for encoding positional information, (3) use grouped query attention (GQA) [1] instead of multi-head attention (MHA), (4) replace the feed forward network (FFN) with SwiGLU FFN [41], (5) use flash attention [13] for computing the scaled dot-product attention, and (6) use the same tokenizer as LLama [46].

Existing LLMs use the same configuration for each transformer layer in the model, resulting in a uniform allocation of parameters across layers. Unlike these models, each transformer layer in OpenELM has a different configuration (*e.g.*, number of heads and feed forward network dimension), resulting in variable number of parameters in each layer of the model. This lets OpenELM to better utilize the available parameter budget for achieving higher accuracies. We implement this non-uniform allocation of parameters across layers using *layer-wise scaling* (also referred as block-wise scaling in [30]).

Layer-wise scaling. A standard transformer layer is composed of multi-head attention (MHA) and feed-forward network (FFN). For non-uniform allocation of parameters in the transformer layer, we adjust the number of attention heads and the FFN multiplier in each transformer layer.

Assume that the standard transformer model with uniform parameter allocation has N transformer layers and the dimensionality of the input to each layer is d_{model}. The MHA has n_h heads and dimension of each head is $d_h = \frac{d_{model}}{n_h}$. Also, the hidden dimension for FFN is $d_{\text{FFN}} = m \cdot d_{model}$, where m is a scalar FFN multiplier.

We introduce parameters α and β to scale the number of attention heads n_h and FFN multiplier m per layer respectively. For the i-th layer, n_h and m are computed as

$$
\begin{aligned}
n_h^i &= \frac{\alpha^i \cdot d_{model}}{d_h}, \quad m^i = \beta^i \\
\text{where } \alpha^i &= \alpha_{min} + \frac{(\alpha_{max} - \alpha_{min}) \cdot i}{N-1}, \\
\text{and } \beta^i &= \beta_{min} + \frac{(\beta_{max} - \beta_{min}) \cdot i}{N-1}, 0 \leq i < N.
\end{aligned} \tag{1}
$$

Here, α_{min} and α_{max} are the hyper-parameters that allow us to scale the attention heads. Similarly, β_{min} and β_{max} let us to vary the width of FFN layers. Therefore, varying the configuration of standard transformer layers using α and β results in non-uniform allocation of parameters in the model. Note, setting $\alpha_{min} = \alpha_{max} = 1.0$ and $m_i = m$ produces the standard uniform transformer model.

2.2. Pre-training data

For pre-training, we use public datasets. Specifically, our pre-training dataset contains RefinedWeb [35], deduplicated PILE [15], a subset of RedPajama [11], and a subset of Dolma v1.6 [42], totaling approximately 1.8 trillion tokens. These details are also summarized in Tab. 2.

On-the-fly tokenization and data filtering. Unlike previous approaches that utilize pre-tokenized data [5, 17], we filter and tokenize text data on-the-fly. This facilitates seamless experimentation with various tokenizers, thereby significantly simplifying prototyping and research endeavors. In our experiments, we use the same tokenizer as used in LLama [46].

To filter out low-length sequences, we apply two filtering methods. The first method operates at the *character-level*, checking if the number of characters in the sequence is below a specified threshold. The second method operates at the *token-level*, where it examines whether the sequence contains fewer tokens than a specified threshold. Sequences that are shorter than either of these thresholds are skipped. In our experiments, we use 200 characters and 256 tokens as character and token-level filtering thresholds.

2.3. Training details

We train OpenELM variants for 350k iterations (or training steps) using CoreNet (formerly CVNets [29]). We use AdamW [28] as an optimizer. We use a cosine learning rate

Source	Subset	Tokens
RefinedWeb		665 B
RedPajama	Github	59 B
	Books	26 B
	ArXiv	28 B
	Wikipedia	24 B
	StackExchange	20 B
	C4	175 B
PILE		207 B
Dolma	The Stack	411 B
	Reddit	89 B
	PeS2o	70 B
	Project Gutenberg	6 B
	Wikipedia + Wikibooks	4.3 B

Table 2. Dataset used for pre-training OpenELM.

Task	Metric
ARC-c	Normalized accuracy
ARC-e	Normalized accuracy
BoolQ	Accuracy
HellaSwag	Normalized accuracy
PIQA	Normalized accuracy
SciQ	Accuracy
WinoGrande	Accuracy

(a) Standard zero-shot metrics

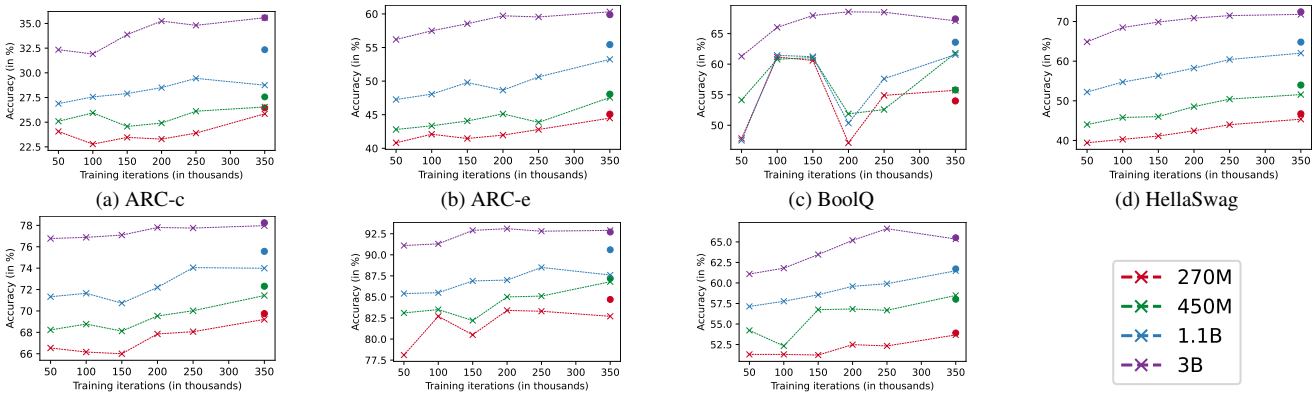
Task	Metric	Num. few shot examples
ARC-c	Normalized accuracy	25
HellaSwag	Normalized accuracy	10
MMLU	Accuracy	5
TruthfulQA-mc2	Accuracy	0
WinoGrande	Accuracy	5

(b) OpenLLM leaderboard

Task	Metric	Num. few shot examples
ARC-c	Normalized accuracy	25
CrowsPairs-En	PCT stereotype	25
HellaSwag	Normalized accuracy	10
WinoGrande	Accuracy	5
MMLU	Accuracy	5
PIQA	Normalized accuracy	0
RACE	Accuracy	0

(c) LLM360

Table 3. Tasks and metrics used for evaluating OpenELM.



(a) ARC-c (b) ARC-e (c) BoolQ (d) HellaSwag

(e) PIQA (f) SciQ (g) WinoGrande OpenELM sizes

Figure 1. **OpenELM's performance across training iterations on standard zero-shot tasks**. In the majority of tasks, the performance of OpenELM shows improvement with increasing training duration. Furthermore, the model checkpoint obtained by averaging the last five checkpoints, collected at intervals of 5k iterations, demonstrates comparable or slightly better performance (indicated by ● markers) as compared to the last checkpoint obtained after 350k iterations.

schedule [27], with warm up of 5k iterations, and decay the final learning rate down to 10% of maximum learning rate. We use a weight decay of 0.1 and gradient clipping of 1.0. We train four variants of OpenELM (270M, 450M, 1.1B, and 3B), and for some, we use FSDP [56] and activation checkpointing [8]. Please refer to Appendix A for additional pre-training details.

2.4. Evaluation details

Following previous works, we evaluate the performance across different tasks using LM Evaluation Harness [16][1]:

- **Standard zero-shot tasks.** We consider 7 standard common-sense reasoning tasks: ARC easy and challenge [10], BoolQ [9], HellaSwag [52], PIQA [6], SciQ [49], and WinoGrande [39].
- **OpenLLM leaderboard tasks.** We use 5 tasks from OpenLLM leaderboard [4]: ARC challenge, HellaSwag, MMLU [20], TruthfulQA [24], and WinoGrande.
- **LLM360 leaderboard tasks.** We use 7 tasks from LLM360 leaderboard [26] for evaluation: ARC challenge, CrowS-Pairs (English version) [32], HellaSwag, WinoGrande, MMLU, PIQA, and RACE [23].

These evaluation frameworks, built on top of LM Evaluation Harness, allows us to comprehensively evaluate OpenELM in terms of reasoning (*e.g.*, ARC-c, HellaSwag, and PIQA), knowledge understanding (*e.g.*, MMLU and RACE), and misinformation & bias (*e.g.*, TruthfulQA and CrowS-Pairs). While there may be some overlap in tasks among these frameworks, they primarily differ in the few-shot settings, as outlined in Tab. 3.

3. Experimental Results

Pre-training results. We evaluate the performance of OpenELM on zero-shot and few-shot settings (Tab. 3). We compare OpenELM with publicly available LLMs, namely PyThia [5], Cerebras-GPT [14], TinyLlama [54], OpenLM [18], MobiLlama [44], and OLMo [17]. The works most closely related to ours are MobiLlama and OLMo. These models are trained on comparable dataset mixtures, with similar or larger number of pre-training tokens.

In Fig. 1, the accuracy of OpenELM is plotted against training iterations for 7 standard zero-shot tasks. We observe an overall increase in accuracy with longer training

[1]We use commit dc90fec of https://github.com/EleutherAI/lm-evaluation-harness

Model	Model size	Pretraining tokens	ARC-c	ARC-e	BoolQ	HellaSwag	PIQA	SciQ	WinoGrande	Average	Average w/o SciQ
OpenELM (Ours)	0.27 B	1.5 T	26.45	45.08	53.98	46.71	69.75	84.70	53.91	54.37	49.31
MobiLlama [44]	0.50 B	1.3 T	26.62	46.04	55.72	51.06	71.11	83.60	53.20	55.34	50.63
OpenELM (Ours)	0.45 B	1.5 T	**27.56**	**48.06**	**55.78**	**53.97**	**72.31**	**87.20**	**58.01**	**57.56**	**52.62**
TinyLlama [54]	1.10 B	3.0 T	30.12	55.25	57.83	59.20	73.29	-	59.12	-	55.80
OpenLM [18]	1.00 B	1.6 T	31.00	56.00	**65.00**	61.00	74.00	-	60.00	-	57.83
MobiLlama [44]	0.80 B	1.3 T	28.84	49.62	60.03	52.45	73.18	85.90	55.96	58.00	53.35
MobiLlama [44]	1.26 B	1.3 T	31.91	56.65	60.34	62.18	74.81	89.10	59.27	62.04	57.53
OLMo [17]	1.18 B	3.0 T	31.06	**57.28**	61.74	62.92	75.14	87.00	59.98	62.16	58.02
OpenELM (Ours)	1.08 B	1.5 T	**32.34**	55.43	63.58	**64.81**	**75.57**	**90.60**	**61.72**	**63.44**	**58.91**
OpenELM (Ours)	3.04 B	1.5 T	35.58	59.89	67.40	72.44	78.24	92.70	65.51	67.39	63.18

(a) Results on zero-shot tasks with respect to the standard metrics defined in Tab. 3a.

Model	Model size	Pretraining tokens	ARC-c	HellaSwag	MMLU	TruthfulQA-mc2	WinoGrande	Average
Cerebras-GPT [14]	0.26 B	5.1 B	22.01	28.99	**26.83**	**45.98**	52.49	35.26
OPT [55]	0.35 B	0.2 T	23.55	36.73	26.02	40.83	52.64	35.95
OpenELM (Ours)	0.27 B	1.5 T	**27.65**	**47.15**	25.72	39.24	**53.83**	**38.72**
Pythia [5]	0.41 B	0.3 T	24.83	41.29	25.99	**40.95**	54.38	37.49
MobiLlama [44]	0.50 B	1.3 T	29.52	52.75	**26.09**	37.55	56.27	40.44
OpenELM (Ours)	0.45 B	1.5 T	**30.20**	**53.86**	26.01	40.18	**57.22**	**41.50**
MobiLlama [44]	0.80 B	1.3 T	30.63	54.17	25.2	38.41	56.35	40.95
Pythia [5]	1.40 B	0.3 T	32.68	54.96	25.56	**38.66**	57.30	41.83
MobiLlama [44]	1.26 B	1.3 T	34.64	63.27	23.87	35.19	60.77	43.55
OLMo [17]	1.18 B	3.0 T	34.47	63.81	26.16	32.94	60.46	43.57
OpenELM (Ours)	1.08 B	1.5 T	**36.69**	**65.71**	**27.05**	36.98	**63.22**	**45.93**
OpenELM (Ours)	3.04 B	1.5 T	42.24	73.28	26.76	34.98	67.25	48.90

(b) Results on OpenLLM Leaderboard tasks with respect to the metrics defined in Tab. 3b.

Model	Model size	Pretraining tokens	ARC-c	CrowS-Pairs	HellaSwag	MMLU	PIQA	RACE	TruthfulQA	WinoGrande	Average
OpenELM (Ours)	0.27 B	1.5 T	27.65	66.79	47.15	25.72	69.75	30.91	39.24	53.83	45.13
MobiLlama [44]	0.50 B	1.3 T	29.52	65.47	52.75	**26.09**	71.11	32.15	37.55	56.27	46.37
OpenELM (Ours)	0.45 B	1.5 T	**30.20**	**68.63**	**53.86**	26.01	**72.31**	**33.11**	**40.18**	**57.22**	**47.69**
MobiLlama [44]	0.80 B	1.3 T	30.63	66.25	54.17	25.2	73.18	33.68	**38.41**	56.35	47.23
MobiLlama [44]	1.26 B	1.3 T	34.64	70.24	63.27	23.87	74.81	35.02	35.19	60.77	49.73
OLMo [17]	1.18 B	3.0 T	34.47	69.95	63.81	26.16	75.14	**36.75**	32.94	60.46	49.96
OpenELM (Ours)	1.08 B	1.5T	**36.69**	**71.74**	**65.71**	**27.05**	**75.57**	36.46	36.98	**63.22**	**51.68**
OpenELM (Ours)	3.04 B	1.5 T	42.24	73.29	73.28	26.76	78.24	38.76	34.98	67.25	54.35

(c) Results on LLM360 tasks with respect to the metrics defined in Tab. 3c.

Table 4. **Comparison of OpenELM with publicly available LLMs across various evaluation frameworks.**. We chose MobiLlama and OLMo as our baselines because they are pre-trained on public datasets using a similar or larger number of tokens. We evaluate OpenELM, MobiLlama, and OLMo using the same LM evaluation harness version. Results for other models in Tab. 4a and Tab. 4b are taken from their official GitHub repositories and the OpenLLM leaderboard [4], respectively. Best task accuracy for each model category is highlighted in bold. Models pre-trained with less data are highlighted in gray color.

durations across most tasks. Additionally, the checkpoint obtained by averaging the last five checkpoints, collected at intervals of 5000 iterations, demonstrates comparable or slightly better accuracy compared to the final checkpoint obtained after 350k iterations. This improvement is likely due to noise reduction through weight averaging. Consequently, we use the averaged checkpoint for our main evaluations in Tab. 4, instruction tuning experiments in Tab. 5, and parameter-efficient tuning experiments in Tab. 6.

The results in Tab. 4 span across various evaluation frameworks, and highlights OpenELM's effectiveness over existing methods. For instance, an OpenELM variant with 1.1 billion parameters achieves 1.28% (Tab. 4a), 2.36% (Tab. 4b), and 1.72% (Tab. 4c) higher accuracy compared to OLMo with 1.2 billion parameters. Remarkably, OpenELM achieves this level of accuracy while using $2\times$ less pre-training data.

Instruction tuning results. We use the cleaned variant of UltraFeedback [3, 12] dataset that consists of 60k prompts for instruction tuning. We do instruction tuning using Alignment Handbook library [47]. For optimization, we use either the statistical rejection sampling method [25] or the direct preference optimization method [37]. These sampling method details along with other hyper-parameters and fine-tuning details are given in Appendix B.

Tab. 5 shows that instruction tuning consistently improves OpenELM's average accuracy by 1-2% across different evaluation frameworks.

Model Size	Instruction Tuned?	ARC-c	ARC-e	BoolQ	HellaSwag	PIQA	SciQ	WinoGrande	Average
0.27 B	✗	26.45	45.08	**53.98**	46.71	69.75	**84.70**	**53.91**	54.37
	✓	**30.55**	**46.68**	48.56	**52.07**	**70.78**	84.40	52.72	**55.11**
0.45 B	✗	27.56	48.06	55.78	53.97	72.31	87.20	58.01	57.56
	✓	**30.38**	**50.00**	**60.37**	**59.34**	**72.63**	**88.00**	**58.96**	**59.95**
1.08 B	✗	32.34	**55.43**	63.58	64.81	**75.57**	**90.60**	61.72	63.44
	✓	**37.97**	52.23	**70.00**	**71.20**	75.03	89.30	**62.75**	**65.50**
3.04 B	✗	35.58	59.89	67.40	72.44	78.24	**92.70**	65.51	67.39
	✓	**39.42**	**61.74**	**68.17**	**76.36**	**79.00**	92.50	**66.85**	**69.15**

(a) Results on zero-shot tasks with respect to the metrics defined in Tab. 3a.

Model Size	Instruction Tuned?	ARC-c	HellaSwag	MMLU	TruthfulQA	WinoGrande	Average
0.27 B	✗	27.65	47.15	25.72	**39.24**	**53.83**	38.72
	✓	**32.51**	**51.58**	**26.70**	38.72	53.20	**40.54**
0.45 B	✗	30.20	53.86	**26.01**	40.18	57.22	41.50
	✓	**33.53**	**59.31**	25.41	**40.48**	**58.33**	**43.41**
1.08 B	✗	36.69	65.71	**27.05**	36.98	63.22	45.93
	✓	**41.55**	**71.83**	25.65	**45.95**	**64.72**	**49.94**
3.04 B	✗	42.24	73.28	**26.76**	34.98	67.25	48.90
	✓	**47.70**	**76.87**	24.80	**38.76**	**67.96**	**51.22**

(b) Results on OpenLLM Leaderboard tasks with respect to the metrics defined in Tab. 3b.

Model Size	Instruction Tuned?	ARC-c	CrowS-Pairs	HellaSwag	MMLU	PIQA	RACE	TruthfulQA	WinoGrande	Average
0.27 B	✗	27.65	**66.79**	47.15	25.72	69.75	30.91	**39.24**	**53.83**	45.13
	✓	**32.51**	66.01	**51.58**	**26.70**	**70.78**	33.78	38.72	53.20	**46.66**
0.45 B	✗	30.20	**68.63**	53.86	**26.01**	72.31	33.11	40.18	57.22	47.69
	✓	**33.53**	67.44	**59.31**	25.41	**72.63**	**36.84**	**40.48**	**58.33**	**49.25**
1.08 B	✗	36.69	**71.74**	65.71	**27.05**	**75.57**	36.46	36.98	63.22	51.68
	✓	**41.55**	71.02	**71.83**	25.65	75.03	**39.43**	**45.95**	**64.72**	**54.40**
3.04 B	✗	42.24	**73.29**	73.28	**26.76**	78.24	**38.76**	34.98	67.25	54.35
	✓	**47.70**	72.33	**76.87**	24.80	**79.00**	38.47	**38.76**	**67.96**	**55.73**

(c) Results on LLM360 tasks with respect to the metrics defined in Tab. 3c.

Table 5. **Instruction tuning improves OpenELM's accuracy across different model sizes.**

Parameter-efficient fine-tuning (PEFT) results. We use the CommonSense reasoning training and evaluation setup [22]. This setup provides 170k training samples across 8 multiple-choice datasets for PEFT studies with different methods, including LoRA [21] and DoRA [51]. We integrate OpenELM with these methods, and finetune the resulting model for three epochs using 8 NVIDIA H100 GPUs. Tab. 6 shows that PEFT methods can be applied to OpenELM. LoRA and DoRA deliver similar accuracy on average across the given CommonSense reasoning datasets.

4. Benchmarking

Hardware. We benchmark on modern, consumer-grade hardware with BFloat16 as the data type. Specifically, CUDA benchmarks were performed on a workstation with an Intel i9-13900KF CPU, equipped with 64 GB of DDR5-4000 DRAM, and an NVIDIA RTX 4090 GPU with 24 GB of VRAM, running Ubuntu 22.04. PyTorch v2.2.2 [34] was used, with the most recent versions of models and the associated libraries. HuggingFace Transformers v4.39.3 [50] was used to benchmark HuggingFace models. We did not use Torch Inductor for model compilation.

To benchmark OpenELM models on the Apple silicon, we used an Apple MacBook Pro with an M2 Max system-on-chip and 64GiB of RAM, running macOS 14.4.1. We ported the code and the weights of OpenELM to Apple MLX v0.10.0 [19]. To maximize the throughput, lazy evaluation was used in MLX with 8 tokens evaluated at a time.

Evaluation. We provide two separate measurements for token throughput (measured in terms of tokens processed per second): (1) prompt processing (pre-fill), and (2) token generation. Additionally, we also report the total combined throughput. We benchmark all models sequentially, and execute one full "dry run" generating 1024 tokens for the first model, since we found that this significantly increases the throughput of generation for subsequent models. Before measurement for each individual model, we warm up the model by executing a single forward pass to allow the frameworks to perform further auto-tuning, if any. In all experiments, we use key-value caching and generate 1024 tokens in addition to the prompt tokens in all tests. Static

Model Size	PEFT	ARC-c	ARC-e	BoolQ	HellaSwag	PIQA	SIQA	WinoGrande	OBQA	Average
0.27 B	LoRA	24.57	26.60	62.14	24.84	50.05	42.02	49.88	28.00	38.51
	DoRA	26.19	28.07	62.20	25.22	50.11	44.42	50.12	31.20	39.69
0.45 B	LoRA	28.67	29.88	62.29	25.85	52.39	49.59	50.91	33.20	41.60
	DoRA	28.33	30.39	62.26	25.12	52.29	49.28	50.83	32.00	41.31
1.08 B	LoRA	45.14	61.11	61.77	77.95	72.31	69.70	61.64	59.20	63.60
	DoRA	44.11	61.49	61.68	78.92	71.38	69.04	64.01	58.80	63.68
3.04 B	LoRA	46.93	66.25	62.48	81.22	75.19	70.62	65.51	58.20	65.80
	DoRA	46.50	66.46	62.35	80.84	75.73	70.83	63.77	58.20	65.59

Table 6. **OpenELM with PEFT.** Both LoRA and DoRA demonstrate comparable performance when OpenELM is finetuned on CommonSense reasoning benchmark. It's important to note that these *fine-tuning results*, obtained using the evaluation setup of LLM-Adapters [22], differ from the results in Tabs. 4 and 5. This is because the results in Tabs. 4 and 5 are obtained under zero- and few-shot settings using LM Evaluation Harness. Note that we did not use social interactions QA (SIQA; [40]) and OpenBookQA (OBQA; [31]) in Tabs. 4 and 5 because of evaluation issues with LLama tokenizer in LM Evaluation Harness (see [45]).

Model	Model size	Throughput (Tokens per second)		
		Prompt	Generation	Total
OPT [55]	0.35 B	6524.17	214.11	220.21
OpenELM (Ours)	0.27 B	6427.27	159.67	165.85
MobiLlama [44]	0.50 B	3423.25	136.35	146.86
OpenELM (Ours)	0.45 B	5211.35	128.46	133.42
MobiLlama [44]	0.80 B	4151.75	126.01	130.08
Pythia [5]	1.40 B	4501.85	139.65	143.83
MobiLlama [44]	1.26 B	4938.29	142.96	147.67
OLMo [17]	1.18 B	7151.65	203.40	209.26
OpenELM (Ours)	1.08 B	3681.73	92.15	95.72
OpenELM (Ours)	3.04 B	2712.56	70.11	72.82

(a) Results on NVIDIA CUDA / Linux.

Model	Throughput (Tokens per second)		
	Prompt	Generation	Total
OpenELM-0.27B	1151.41	212.40	218.45
OpenELM-0.27B-4bit	803.99	256.35	262.70
OpenELM-0.45B	910.61	147.26	151.57
OpenELM-0.45B-4bit	883.19	197.81	203.16
OpenELM-1.08B	508.56	78.72	81.04
OpenELM-1.08B-4bit	554.17	117.90	121.14
OpenELM-3.04B-bf16	234.96	33.96	34.97
OpenELM-3.04B-bf16-4bit	211.32	60.33	61.83

(b) Results for the MLX port on Apple macOS.

Table 7. **Benchmark measurements of OpenELM compared to other similar LLMs in its class.**. On CUDA, we evaluate OpenELM, MobiLlama, and OLMo using the CoreNet version of OpenELM and HuggingFace for the other two. On macOS, we only provide results for the MLX version of OpenELM.

key-value cache was used whenever supported. The same prompt was used for all runs, resulting in prompt lengths of 35-36 tokens (depending on the tokenizer).

Results. Tabs. 7a and 7b shows the benchmarking results on GPU and MacBook Pro respectively. Despite OpenELM's higher accuracy for a similar parameter count, we observe that it is slower than OLMo. While the primary focus of this study is reproducibility rather than inference performance, we did comprehensive profiling to understand the bottlenecks. Our analysis reveals that a significant portion of OpenELM's processing time can be attributed to our naive implementation of RMSNorm (Tab. 8). Specifically, naive RMSNorm implementation results in many individual kernel launches each of which processes a small input, rather than a launch of a single, fused kernel, as would be the case with *e.g.* LayerNorm. By replacing the naive RMSNorm with Apex's RMSNorm [33], we observe a notable increase in OpenELM's throughput. However, a substantial performance gap persists compared to the models that use optimized LayerNorm, in part because (1) OpenELM has 113 RMSNorm layers as compared to 33 LayerNorm layers in OLMo and (2) Apex's RMSNorm is not optimized for small inputs. To further illustrate the performance degradation attributable to RMSNorm, we replaced the LayerNorm in OLMo with RMSNorm, and observed a significant drop in generation throughput. In future work, we plan to explore optimization strategies to further improve the inference efficiency of OpenELM.

Model	Normalization layer (# Invocations per token)	Throughput (Tokens per second)		
		Prompt	Generation	Total
OLMo	LayerNorm (33)	7151.65	203.40	209.26
	RMSNorm-Naive (33)	5360.56	171.41	176.92
OpenELM (Ours)	LayerNorm (113)	4697.50	130.34	135.38
	RMSNorm-Naive (113)	3681.73	92.15	95.72
	RMSNorm-Apex (113)	4280.66	113.42	117.81

Table 8. **Normalization layers are a bottleneck.** The throughput of both OLMo-1.18B and OpenELM-1.08B significantly decreases with the naive implementation of RMSNorm in PyTorch compared to highly optimized LayerNorm [2]. Although Apex's [33] RMSNorm implementation leads to notable throughput improvements compared to the naive implementation, a considerable performance gap persists in comparison to LayerNorm. This highlights the substantial optimization potential for future endeavors. The number of invocations per token for each normalization layer is indicated next to the layer name in brackets.

5. Conclusion

This work releases OpenELM, a decoder-only transformer-based open language model. The OpenELM uses a layer-wise scaling method for efficient parameter allocation within the transformer model, resulting in improved accuracy compared to existing models. Additionally, we have made the entire framework open, including training logs, multiple checkpoints, pre-training configurations, and MLX inference code. This extensive release aims to empower and strengthen the open research community, facilitating future research efforts.

Author Contributions

The OpenELM project was led by Sachin Mehta, with additional lead contributions from Mohammad Rastegari and Peter Zatloukal. OpenELM would not have been possible without the help of our many teammates and collaborators. We list author contributions below:

Pre-training dataset collection and tooling: Sachin Mehta and Mohammad Sekhavat

Architecture design: Sachin Mehta

Model training: Sachin Mehta and Mohammad Sekhavat

Evaluation suite and tooling: Sachin Mehta, Qingqing Cao, Mohammad Sekhavat, Mahyar Najibi, Maxwell Horton, and Iman Mirzadeh.

Huggingface integration: Qingqing Cao

Instruction tuning: Qingqing Cao

Parameter-efficient finetuning: Maxwell Horton

Performance analysis and MLX conversion: Chenfan Sun, Dmitry Belenko, and Mahyar Najibi

Code review, bug fixes, and maintenance: Sachin Mehta, Maxwell Horton, Mohammad Shekhavat, and Yanzi Jin

Acknowledgements

We extend our gratitude to the following people for discussions and assistance: Farzad Abdolhosseini, David Harrison, Mehrdad Farajtabar, Fartash Faghri, Oncel Tuzel, Hadipour Ansari, Raviteja Vemulapalli, Aseem Wadhwa, Kumari Nishu, Danny Tormoen, Minsik Cho, Jason Ramapuram, Rich Moe, Arsalan Farooq, Dom L'Eplattenier, Mayank Goel, Hassan Babaie, Chong Wang, Ruoming Pang, Tom Gunter, Antonie Lin, Irina Belousova, and Joris Pelemans.

Broader Impact

The release of OpenELM models aims to empower and enrich the open research community by providing access to state-of-the-art language models. Trained on publicly available datasets, these models are made available without any safety guarantees. Consequently, there exists the possibility of these models producing outputs that are inaccurate, harmful, biased, or objectionable in response to user prompts. Thus, it is imperative for users and developers to undertake thorough safety testing and implement appropriate filtering mechanisms tailored to their specific requirements.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023. 2

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 6

[3] Alvaro Bartolome, Gabriel Martin, and Daniel Vila. Notus. https://github.com/argilla-io/notus, 2023. 4

[4] Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. Open llm leaderboard. https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard, 2023. 1, 3, 4

[5] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023. 1, 2, 3, 4, 6

[6] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020. 3

[7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1

[8] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016. 3

[9] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019. 3

[10] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. 3

[11] Together Computer. Redpajama: An open source recipe to reproduce llama training dataset, 2023. 2

[12] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with high-quality feedback, 2023. 4

[13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022. 2

[14] Nolan Dey, Gurpreet Gosal, Hemant Khachane, William Marshall, Ribhu Pathria, Marvin Tom, Joel Hestness, et al. Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster. *arXiv preprint arXiv:2304.03208*, 2023. 3, 4

[15] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. 2

[16] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, Sept. 2021. 3

[17] Dirk Groeneveld, Iz Beltagy, Pete Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Harsh Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, et al. Olmo: Accelerating the science of language models. *arXiv preprint arXiv:2402.00838*, 2024. 1, 2, 3, 4, 6

[18] Suchin Gururangan, Mitchell Wortsman, Samir Yitzhak Gadre, Achal Dave, Maciej Kilian, Weijia Shi, Jean Mercat, Georgios Smyrnis, Gabriel Ilharco, Matt Jordan, Reinhard Heckel, Alex Dimakis, Ali Farhadi, Vaishaal Shankar, and Ludwig Schmidt. OpenLM: A minimal but performative language modeling (lm) repository, 2023. GitHub repository. 3, 4

[19] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon, 2024. 5

[20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. 3

[21] J. Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *ArXiv*, abs/2106.09685, 2021. 5

[22] Zhiqiang Hu, Yihuai Lan, Lei Wang, Wanyu Xu, Ee-Peng Lim, Roy Ka-Wei Lee, Lidong Bing, and Soujanya Poria. Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. *ArXiv*, abs/2304.01933, 2023. 5, 6

[23] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*, 2017. 3

[24] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021. 3

[25] Tianqi Liu, Yao Zhao, Rishabh Joshi, Misha Khalman, Mohammad Saleh, Peter J. Liu, and Jialu Liu. Statistical Rejection Sampling Improves Preference Optimization, Jan. 2024. arXiv:2309.06657 [cs]. 4

[26] Zhengzhong Liu, Aurick Qiao, Willie Neiswanger, Hongyi Wang, Bowen Tan, Tianhua Tao, Junbo Li, Yuqi Wang, Suqi Sun, Omkar Pangarkar, et al. Llm360: Towards fully transparent open-source llms. *arXiv preprint arXiv:2312.06550*, 2023. 3

[27] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016. 3

[28] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 2

[29] Sachin Mehta, Farzad Abdolhosseini, and Mohammad Rastegari. Cvnets: High performance library for computer vision. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 7327–7330, 2022. 2

[30] Sachin Mehta, Marjan Ghazvininejad, Srinivasan Iyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Delight: Deep and light-weight transformer. *arXiv preprint arXiv:2008.00623*, 2020. 1, 2

[31] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018. 6

[32] Nikita Nangia, Clara Vania, Rasika Bhalerao, and Samuel R Bowman. Crows-pairs: A challenge dataset for measuring social biases in masked language models. *arXiv preprint arXiv:2010.00133*, 2020. 3

[33] NVIDIA Corporation. Apex: A pytorch extension with tools for mixed precision training and more. GitHub, 2024. 6

[34] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. 5

[35] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*, 2023. 2

[36] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016. 10

[37] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, Dec. 2023. arXiv:2305.18290 [cs]. 4

[38] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020. 10

[39] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021. 3

[40] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan LeBras, and Yejin Choi. Socialiqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019. 6

[41] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. 2

[42] Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, Jennifer Dumas, Yanai Elazar, et al. Dolma: An open corpus of three trillion tokens for language model pretraining research. *arXiv preprint arXiv:2402.00159*, 2024. 2

[43] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. 2

[44] Omkar Thawakar, Ashmal Vayani, Salman Khan, Hisham Cholakal, Rao M Anwer, Michael Felsberg, Tim Baldwin, Eric P Xing, and Fahad Shahbaz Khan. Mobillama: Towards accurate and lightweight fully transparent gpt. *arXiv preprint arXiv:2402.16840*, 2024. 1, 3, 4, 6

[45] Hsu Wan Ting. Accuracy not matched for llama1-7b. GitHub issue, 2024. https://github.com/EleutherAI/lm-evaluation-harness/issues/1294. 6

[46] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1, 2

[47] Lewis Tunstall, Edward Beeching, Nathan Lambert, Nazneen Rajani, Shengyi Huang, Kashif Rasul, Alexander M. Rush, and Thomas Wolf. The alignment handbook. https://github.com/huggingface/alignment-handbook, 2023. 4

[48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1

[49] Johannes Welbl, Nelson F Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. *arXiv preprint arXiv:1707.06209*, 2017. 3

[50] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, Oct. 2020. Association for Computational Linguistics. 5

[51] Shih yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. *ArXiv*, abs/2402.09353, 2024. 5

[52] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019. 3

[53] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019. 2

[54] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024. 3, 4

[55] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022. 1, 4, 6

[56] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023. 3

A. Pre-training hyper-parameters

The pre-training hyper-parameters for different OpenELM configurations are given in Tab. 9.

	270M	**450M**	**1.1B**	**3B**
Dimension d_{model}	1280	1536	2048	3072
Num. of layers N	16	20	28	36
Head dimension d_h	64	64	64	128
$\alpha_{min}, \alpha_{max}$ (Eq. (1))	0.5, 1.0			
β_{min}, β_{max} (Eq. (1))	0.5, 4.0			
Normalization layer	RMSNorm			
Positional embeddings	RoPE			
Attention variant	Grouped query attention			
Activation	SwiGLU			
Context length	2048			
Batch size (tokens)	approx. 4M			
Weight tying [36]	yes			
Warm-up iterations	5,000			
Training steps	350,000			
Warm-up init. LR	0.000001			
Max. LR	0.0053	0.0039	0024	0.0012
Min. LR	10% of the max. LR			
Loss function	Cross-entropy			
Optimizer	AdamW (β_1=0.9, β_2=0.95, $\epsilon = 1.e-8$)			
Weight decay	0.1			
Activation checkpointing	✗	✓	✓	✓
FSDP	✗	✗	✗	✓
GPUs	128	128	128	128
GPU Type	A100	H100	A100	H100
GPU Memory	80 GB	80 GB	80 GB	80 GB
Training time (in days)	3	3	11	13

Table 9. Pre-training details for different variants of OpenELM.

B. Hyper-parameters for instruction tuning

We conducted a grid search to determine optimal values for the learning rate and training epochs. For the learning rate, we explored values in the range of [2e-5, 3e-5, 5e-5, 8e-5, 1e-4], while for training epochs, we investigated the range of [3, 5, 8, 10]. The final recipe selected is the one that yielded the highest average accuracy across various tasks as presented in Tab. 3a and Tab. 3c.

We finetune all the models with BFloat16 as a data type. We use activation checkpointing along with gradient accumulation with a step size of two. We use the AdamW optimizer with default beta values. We use the cosine learning rate scheduler with a warm-up ratio of 0.1, and we set the weight decay to 0 and loss temperature beta to 0.01. We set the maximum context length to 1024 and maximum prompt length to 512. Other hyper-parameters are included in Tab. 10.

	270M	**450M**	**1.1B**	**3B**
Batch size	8			
Training epochs	5	8	5	10
Learning rate	2e-5	3e-5	5e-5	1e-4
Loss function	hinge	hinge	sigmoid	hinge
DeepSpeed Zero3 [38]	✗	✓	✓	✓
GPUs	8			
GPU Type	A100	A100	A100	A100
GPU Memory	40 GB	40 GB	40 GB	80 GB
Training time (in hours)	2.5	4.3	6.6	14.2

Table 10. Instruction tuning details for different variants of OpenELM.